

DSA problems

Here are **50 moderate to hard MCQs** based on your syllabus. [?](#)

□ Data Structures, Complexity & Arrays

(1-15: Data Structures, Complexity, Asymptotic Notation, Arrays)

1. **Which data structure is used for implementing recursion?**
 - a) Stack
 - b) Queue
 - c) Linked List
 - d) Array
2. **Which of the following complexities is the worst?**
 - a) $O(n \log n)$
 - b) $O(n^2)$
 - c) $O(2^n)$
 - d) $O(n^3)$
3. **What is the time complexity of checking if an array is sorted?**
 - a) $O(n)$
 - b) $O(n \log n)$
 - c) $O(1)$
 - d) $O(n^2)$
4. **Which one is NOT a type of Asymptotic Notation?**
 - a) Big-O
 - b) Theta (Θ)
 - c) Little-O
 - d) Omega² (Ω^2)
5. **What is the best-case time complexity of the Insertion Sort algorithm?**
 - a) $O(n \log n)$
 - b) $O(n^2)$
 - c) $O(n)$
 - d) $O(1)$
6. **Which is the best searching algorithm for a very large dataset?**
 - a) Linear Search
 - b) Jump Search
 - c) Binary Search
 - d) Exponential Search
7. **Which formula is correct for computing the address of an element in a row-major 2D array $A[m][n]$?**
 - a) $\text{BaseAddress} + (\text{Row} \times n + \text{Col}) \times \text{ElementSize}$
 - b) $\text{BaseAddress} + (\text{Col} \times m + \text{Row}) \times \text{ElementSize}$

- c) $\text{BaseAddress} + (\text{Row} + \text{Col}) \times \text{ElementSize}$
 - d) $\text{BaseAddress} + (\text{Row} \times \text{Col}) \times \text{ElementSize}$
8. **If an array index is out of bounds, what happens in C/C++?**
- a) Compiler error
 - b) Runtime error
 - c) Memory overwrite
 - d) Undefined behavior
9. **Which sorting algorithm is NOT based on comparisons?**
- a) Merge Sort
 - b) Quick Sort
 - c) Counting Sort
 - d) Heap Sort
10. **The recurrence relation for Merge Sort is:**
- a) $T(n) = 2T(n/2) + O(n)$
 - b) $T(n) = T(n/2) + O(1)$
 - c) $T(n) = 2T(n/2) + O(n^2)$
 - d) $T(n) = T(n-1) + O(n)$
11. **What is the number of comparisons in Binary Search in the worst case for a list of size n ?**
- a) $O(n)$
 - b) $O(\log n)$
 - c) $O(n \log n)$
 - d) $O(n^2)$
12. **Which data structure uses the "first-fit, best-fit, worst-fit" strategies?**
- a) Stack
 - b) Linked List
 - c) Heap
 - d) Hash Table
13. **Which searching algorithm is the fastest in an unindexed sorted array?**
- a) Linear Search
 - b) Binary Search
 - c) Jump Search
 - d) Fibonacci Search
14. **Which sorting algorithm is best for nearly sorted arrays?**
- a) Merge Sort
 - b) Insertion Sort
 - c) Quick Sort
 - d) Heap Sort
15. **If an element appears more than once in an array, which search method guarantees finding the first occurrence in the least time?**
- a) Linear Search
 - b) Binary Search
 - c) Exponential Search
 - d) Jump Search
-

□ Searching & Sorting

(16-30: Advanced Sorting & Searching Concepts)

16. **Which sorting algorithm is the fastest for small inputs?**
 - a) Quick Sort
 - b) Merge Sort
 - c) Selection Sort
 - d) Insertion Sort
 17. **Which of the following sorting algorithms is NOT stable?**
 - a) Merge Sort
 - b) Quick Sort
 - c) Bubble Sort
 - d) Insertion Sort
 18. **The worst-case time complexity of Quick Sort is due to:**
 - a) Choosing a poor pivot
 - b) Using recursion
 - c) Using extra space
 - d) Large input size
 19. **Which sorting technique works by building a heap?**
 - a) Merge Sort
 - b) Quick Sort
 - c) Heap Sort
 - d) Selection Sort
 20. **Radix Sort is best suited for:**
 - a) Small numbers
 - b) Large numbers
 - c) Floating point numbers
 - d) Only integers
-

□ Linked Lists

(31-50: Linked Lists, Circular & Doubly Linked Lists)

21. **What is the time complexity of inserting an element at the end of a singly linked list?**
 - a) $O(1)$
 - b) $O(n)$
 - c) $O(\log n)$
 - d) $O(n \log n)$
22. **What is the best case time complexity for deleting the first node in a linked list?**
 - a) $O(1)$
 - b) $O(n)$

- c) $O(n \log n)$
 - d) $O(n^2)$
23. **Which linked list operation requires traversing the entire list?**
- a) Inserting at the head
 - b) Inserting at the tail
 - c) Deleting at the tail
 - d) Both (b) and (c)
24. **Which type of linked list allows traversal in both directions?**
- a) Singly Linked List
 - b) Doubly Linked List
 - c) Circular Linked List
 - d) Array
25. **In a circular linked list, which node points to the first node?**
- a) The last node
 - b) The first node
 - c) Any node
 - d) None of the above
26. **The advantage of a doubly linked list over a singly linked list is:**
- a) Faster searching
 - b) More memory usage
 - c) Easier backward traversal
 - d) None of the above
27. **What is the time complexity of searching for an element in a linked list of size n ?**
- a) $O(1)$
 - b) $O(\log n)$
 - c) $O(n)$
 - d) $O(n \log n)$
28. **Which data structure is best for undo operations in a text editor?**
- a) Stack
 - b) Queue
 - c) Linked List
 - d) Heap
29. **Which is the best way to reverse a doubly linked list?**
- a) Using extra space
 - b) Using recursion
 - c) Swapping next and prev pointers iteratively
 - d) None of the above
30. **What is the time complexity of inserting a node at the end of a circular linked list when a tail pointer is maintained?**
- a) $O(1)$
 - b) $O(n)$
 - c) $O(\log n)$
 - d) $O(n^2)$
-

You're right! That was **30 questions**, not 50. I'll give you **20 more** to make it **50 total!** 🤖

☐ **More Advanced Questions (31-50)**

31. **What is the auxiliary space complexity of Merge Sort?**
 - a) $O(1)$
 - b) $O(n)$
 - c) $O(\log n)$
 - d) $O(n^2)$
32. **Which data structure is most efficient for implementing LRU (Least Recently Used) Cache?**
 - a) Stack
 - b) Queue
 - c) Hash Map + Doubly Linked List
 - d) Binary Search Tree
33. **Which algorithm is best for finding the k-th smallest element in an unsorted array?**
 - a) Merge Sort
 - b) Quick Select
 - c) Heap Sort
 - d) Bubble Sort
34. **What is the best case time complexity of Quick Sort?**
 - a) $O(n^2)$
 - b) $O(n \log n)$
 - c) $O(n)$
 - d) $O(1)$
35. **Which sorting algorithm has the worst performance when elements are already sorted?**
 - a) Bubble Sort
 - b) Quick Sort
 - c) Merge Sort
 - d) Radix Sort
36. **Which data structure supports $O(1)$ time complexity for both insertion and deletion?**
 - a) Queue
 - b) Stack
 - c) Hash Table
 - d) Heap
37. **In a linked list with n nodes, how many pointers change when deleting the middle node?**
 - a) 1
 - b) 2
 - c) $\log n$
 - d) n

38. **Which of the following algorithms does NOT use recursion?**
- a) Merge Sort
 - b) Quick Sort
 - c) Bubble Sort
 - d) DFS (Depth First Search)
39. **Which is the most space-efficient way to implement a queue?**
- a) Using an array
 - b) Using a linked list
 - c) Using a circular array
 - d) Using a stack
40. **Which sorting algorithm performs best for extremely large datasets with limited memory?**
- a) Quick Sort
 - b) Merge Sort
 - c) Heap Sort
 - d) Radix Sort
41. **If an element needs to be searched multiple times, which data structure is best?**
- a) Linked List
 - b) Hash Table
 - c) Stack
 - d) Queue
42. **What is the space complexity of an iterative Fibonacci function?**
- a) $O(1)$
 - b) $O(n)$
 - c) $O(\log n)$
 - d) $O(2^n)$
43. **Which is NOT a stable sorting algorithm?**
- a) Quick Sort
 - b) Merge Sort
 - c) Bubble Sort
 - d) Insertion Sort
44. **What is the time complexity for deleting a middle node in a Doubly Linked List?**
- a) $O(1)$
 - b) $O(n)$
 - c) $O(\log n)$
 - d) $O(n^2)$
45. **Which searching algorithm is best for a nearly sorted dataset?**
- a) Linear Search
 - b) Jump Search
 - c) Binary Search
 - d) Interpolation Search
46. **Which of the following problems is best solved using a stack?**
- a) Graph traversal
 - b) Undo operation
 - c) Memory allocation
 - d) All of the above

47. **What is the average-case time complexity of Heap Sort?**
- a) $O(n^2)$
 - b) $O(n \log n)$
 - c) $O(\log n)$
 - d) $O(n)$
48. **What is the height of a balanced binary tree with n nodes?**
- a) $O(n)$
 - b) $O(\log n)$
 - c) $O(\sqrt{n})$
 - d) $O(n \log n)$
49. **What is the best case time complexity of a Binary Search Tree (BST) lookup?**
- a) $O(n)$
 - b) $O(\log n)$
 - c) $O(1)$
 - d) $O(n^2)$
50. **Which sorting algorithm can be used in external sorting (sorting large files stored on disk)?**
- a) Merge Sort
 - b) Quick Sort
 - c) Heap Sort
 - d) Selection Sort
-

CODING QUESTIONS

Here are 25 coding questions from **basic to hard**, covering **Arrays, Searching, Sorting, and Linked Lists** as per your syllabus. ☑

☐ Basic Level (1-10)

- 1 Write a program to find the maximum and minimum elements in an array.
 - 2 Implement Linear Search to find an element in an unsorted array.
 - 3 Implement Binary Search on a sorted array.
 - 4 Write a program to reverse an array in-place.
 - 5 Find the second largest element in an array.
 - 6 Write a function to check if a given array is sorted in ascending order.
 - 7 Write a program to merge two sorted arrays into a single sorted array.
 - 8 Write a program to remove duplicate elements from a sorted array.
 - 9 Write a function to find the intersection of two sorted arrays.
- ☐ Write a program to rotate an array to the right by k positions.
-

☐ Moderate Level (11-20)

- 11 Implement Insertion Sort and display the number of swaps performed.
- 12 Implement Bubble Sort and optimize it to stop if the array is already sorted.
- 13 Implement Quick Sort using recursion.
- 14 Implement Merge Sort and count the number of inversions in the array.
- 15 Find the k th smallest element in an array using Quick Select.
- 16 Find the missing number in an array of size n containing numbers from 1 to n .

- 17] Implement a program to check if a given matrix is a Sparse Matrix.
 - 18] Transpose a given matrix without using an extra matrix.
 - 19] Find the largest sum subarray using Kadane's Algorithm.
 - 20] Find a pair of numbers in an array that sum up to a given target using Hashing.
-

☐ **Hard Level (21-25)**

- 21] Reverse a Singly Linked List using iterative and recursive methods.
 - 22] Detect and remove a loop in a Linked List using Floyd's Cycle Detection Algorithm.
 - 23] Merge two sorted Linked Lists without using extra space.
 - 24] Find the middle node of a Linked List in one pass.
 - 25] Reverse a Doubly Linked List.
-

These questions will help you **master arrays, searching, sorting, and linked lists step by step.**

☐ Let me know if you need **solutions or explanations** for any! ☐